

Recurrent Neural Network and Attention Mechanism

Md Shad Akhtar

shad.akhtar@iiitd.ac.in

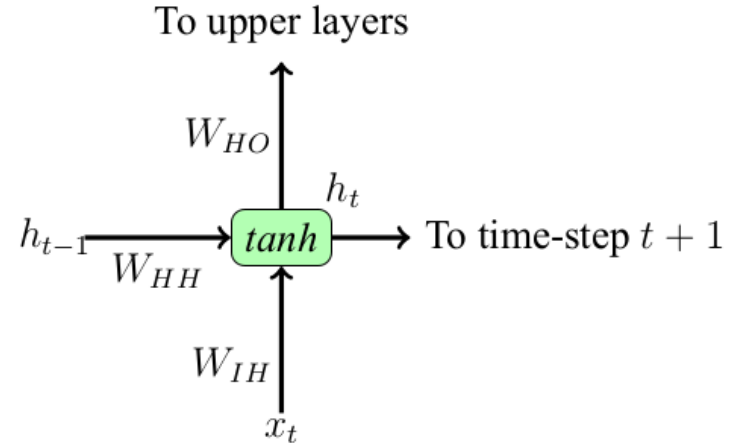


INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**

Recurrent Neural Network (RNN)

- A neural network with feedback connections
- Enable networks to do temporal processing
- Good at learning sequences
- Acts as memory unit

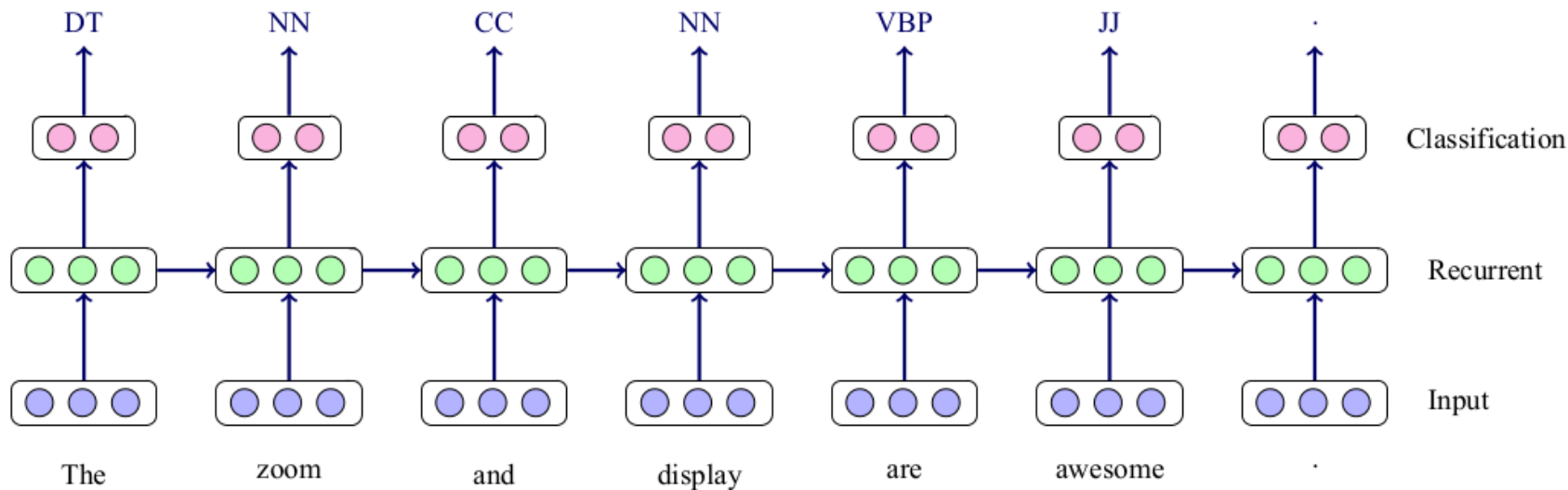
$$\begin{aligned}h_t &= \tanh(W_{IH} \cdot x_t + W_{HH} \cdot h_{t-1} + b) \\ &= \tanh([W_{IH}, W_{HH}] \cdot [x_t, h_{t-1}] + b)\end{aligned}$$



RNN - Example 1

Part-of-speech tagging:

- Given a sentence X, tag each word its corresponding grammatical class.



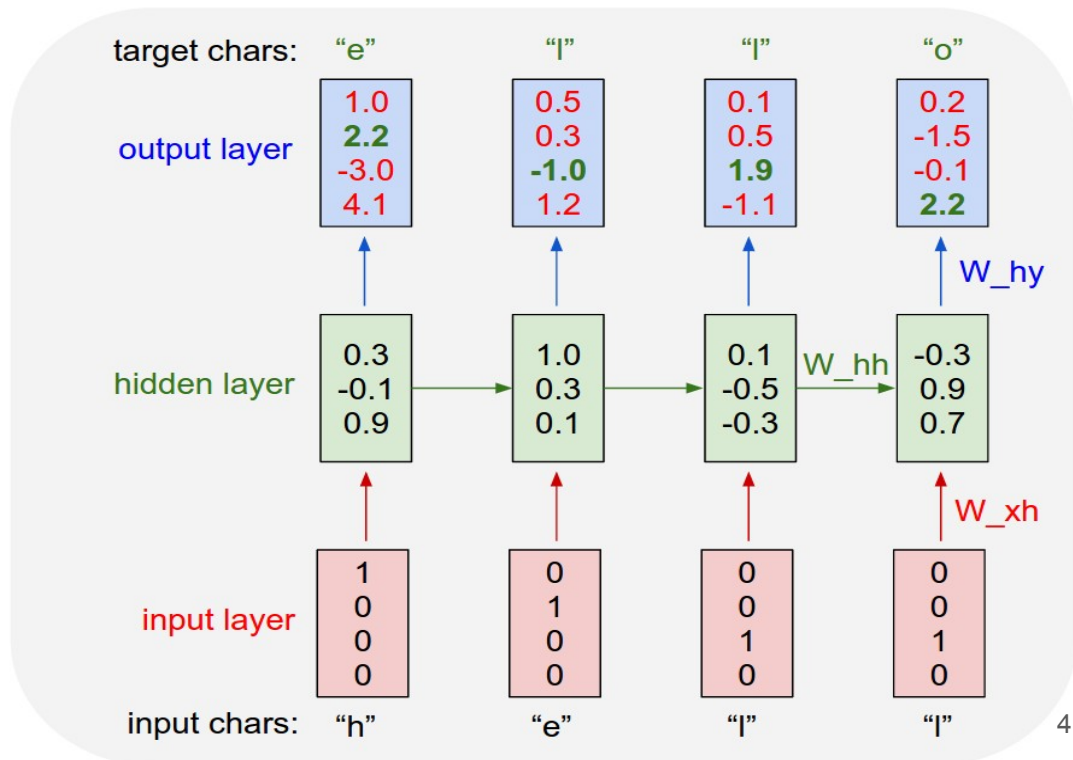
RNN - Example 2

Character level language model:

- Given previous and current characters, predict the next character in the sequence.

Let

- Vocabulary: [h,e,l,o]
- One-hot representations
 - $h = [1 \ 0 \ 0 \ 0]$
 - $e = [0 \ 1 \ 0 \ 0]$
 - $l = [0 \ 0 \ 1 \ 0]$
 - $o = [0 \ 0 \ 0 \ 1]$

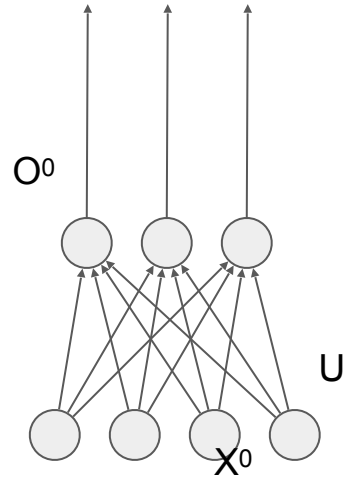


Visualization of RNN through Feed-Forward Neural Network

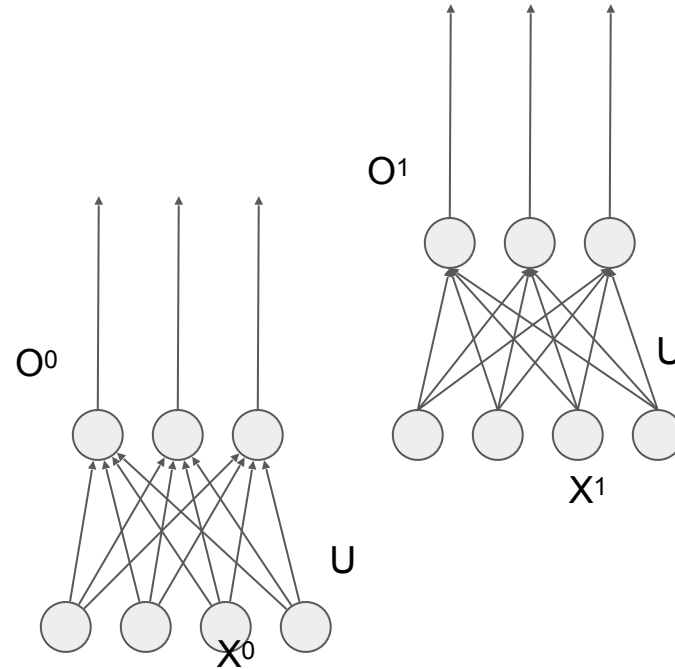
Problem, Data and Network Architecture

- Problem:
 - I/p sequence (X) : $X^0, X^1, \dots, X^T$
 - O/p sequence (O) : $O^0, O^1, \dots, O^T$
- Representation of data:
 - I/p dimension : 4
 - $X^0 \rightarrow 0\ 1\ 1\ 0$
 - O/p dimension : 3
 - $O^0 \rightarrow 0\ 0\ 1$
- Network Architecture
 - Number of neurons at I/p layer : 4
 - Number of neurons at O/p layer : 3
 - Do we need hidden layers?
 - If yes, number of neurons at each hidden layers

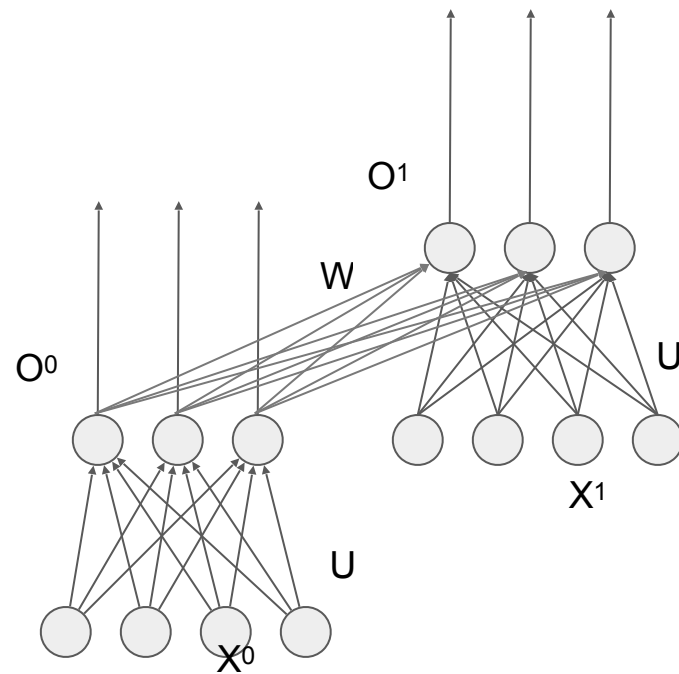
Network @ $t = 0$



Network @ $t = 1$

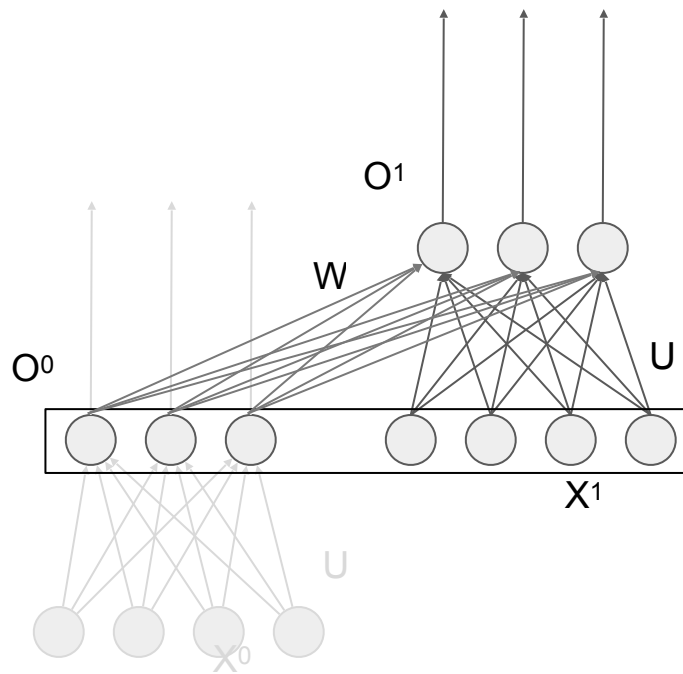


Network @ $t = 1$



Network @ $t = 1$

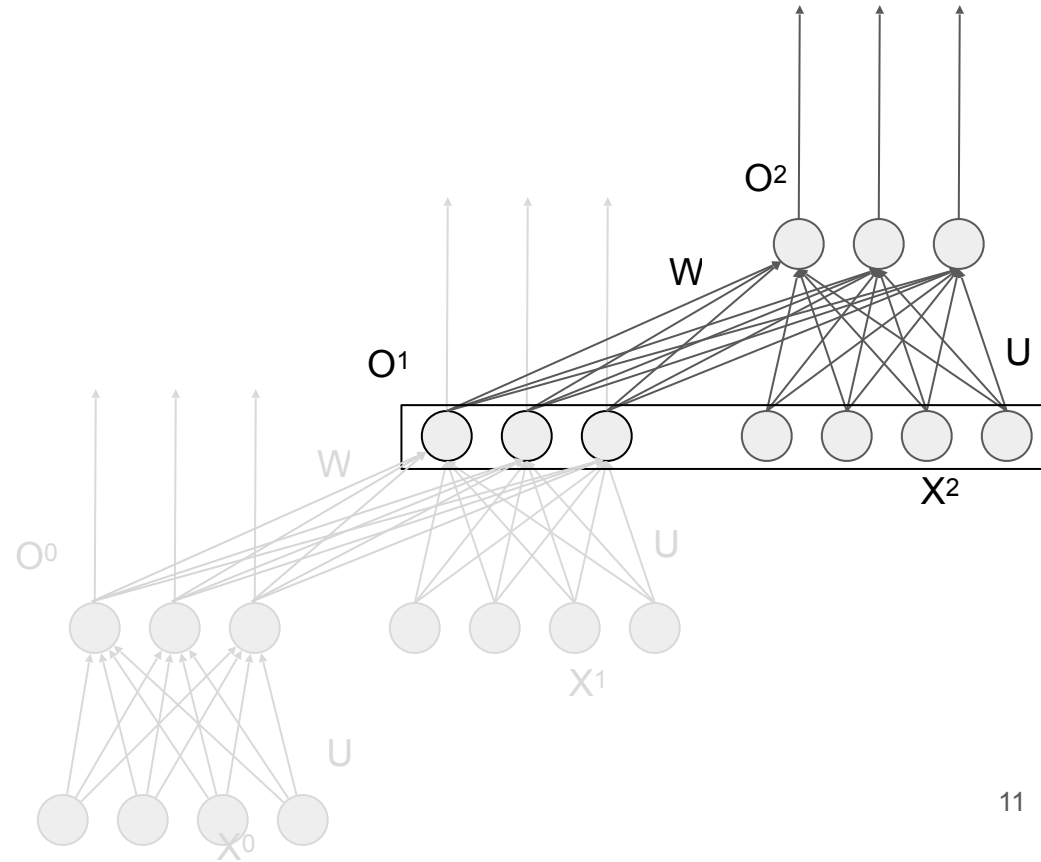
$$\begin{aligned} O^1 &= f(W.O^0 + U.X^1) \\ &= f([W, U] \cdot [O^0, x^1]) \end{aligned}$$



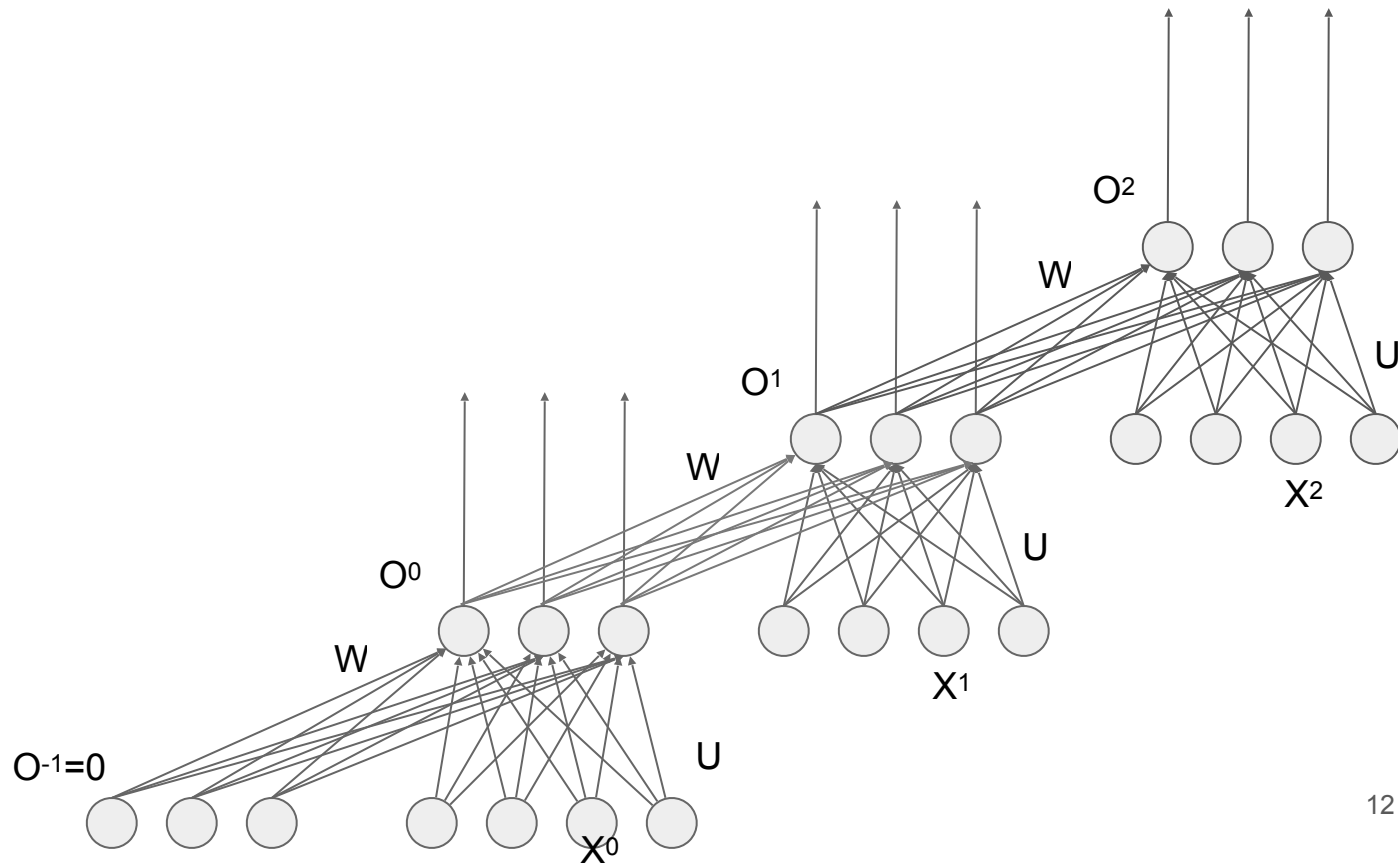
Network @ $t = 2$

t

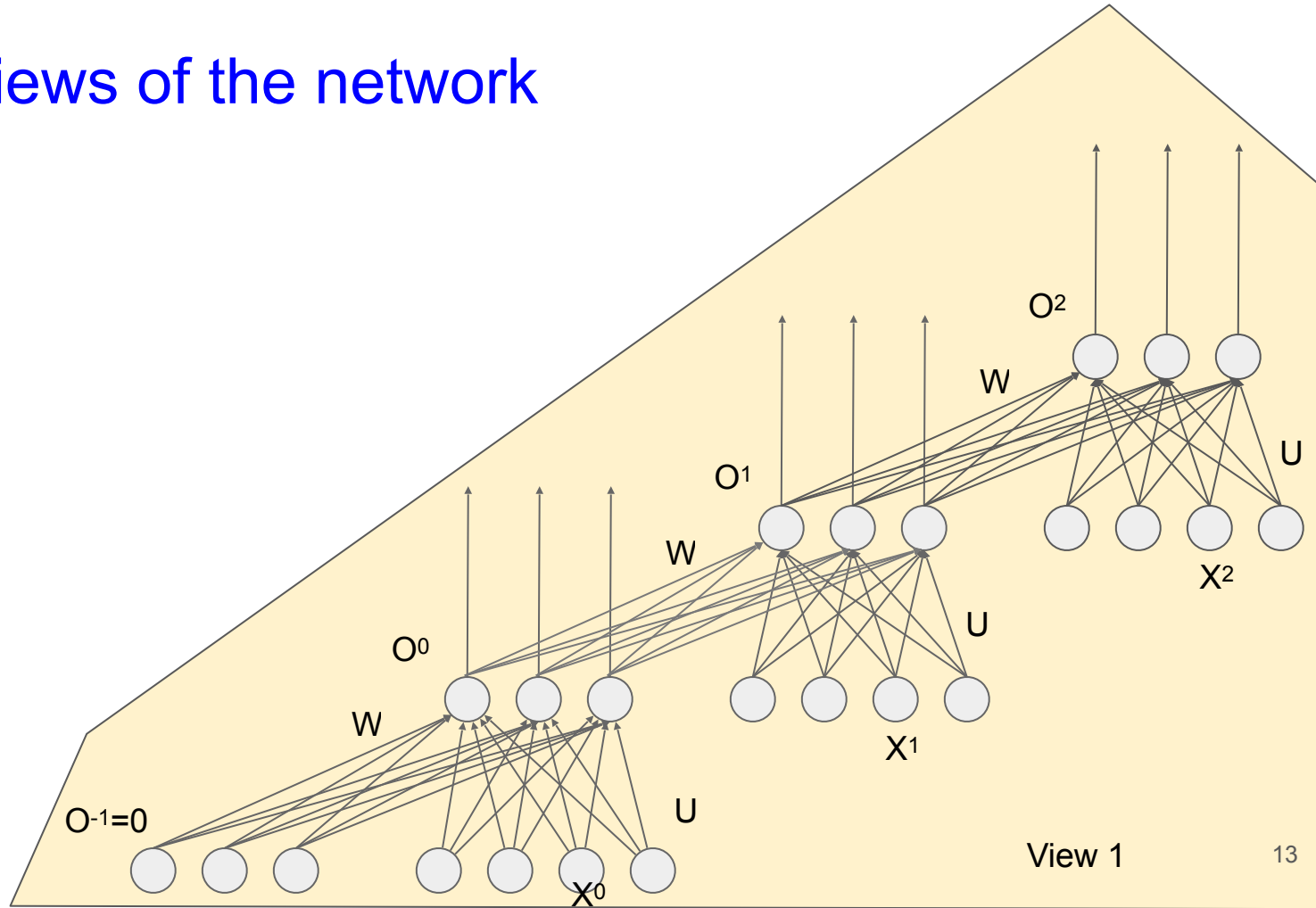
$O^2 = f(W \cdot O^1 + U \cdot X^2)$
 $= f([W, U] \cdot [O^1, x^2])$



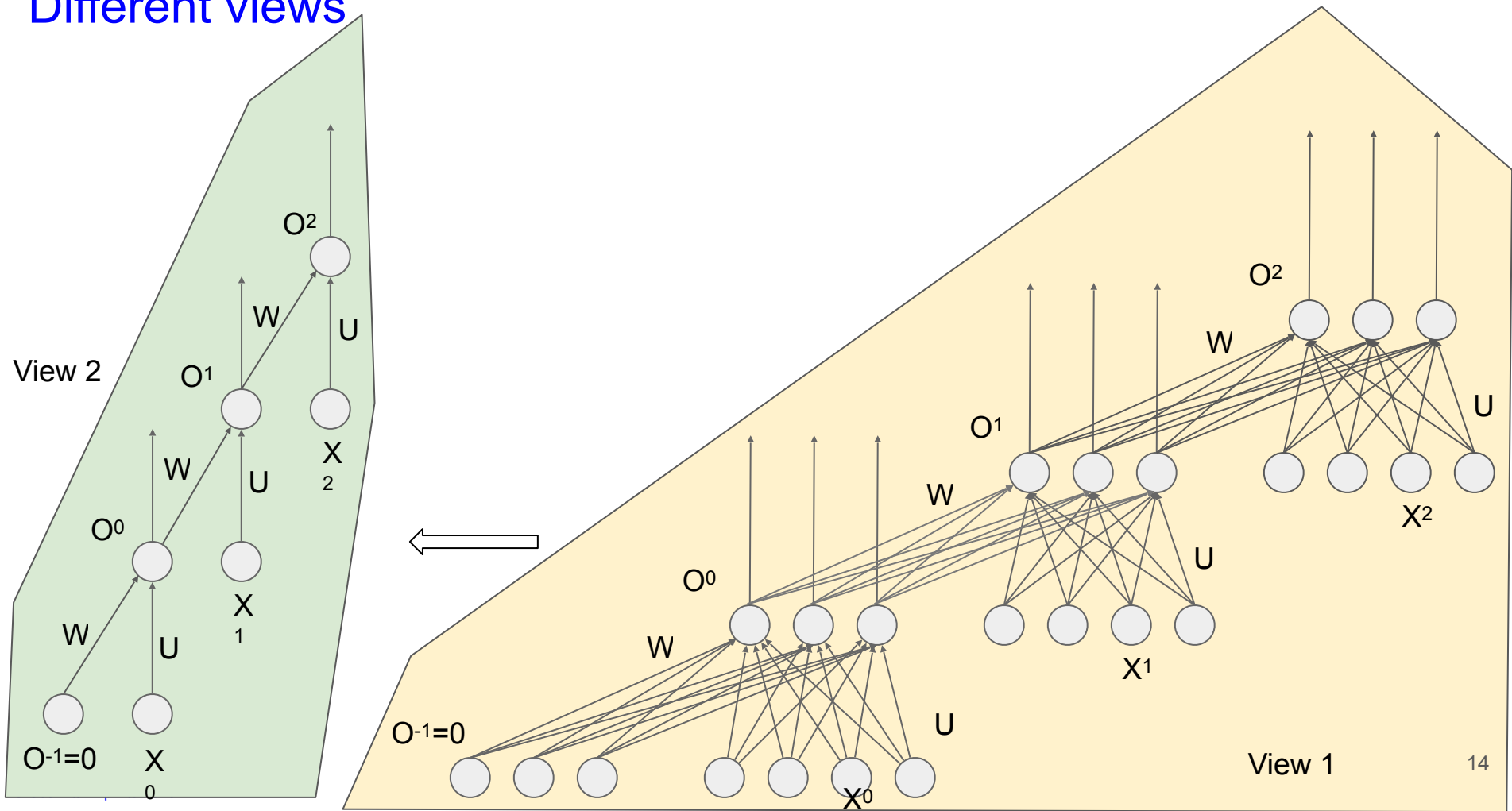
Complete Network



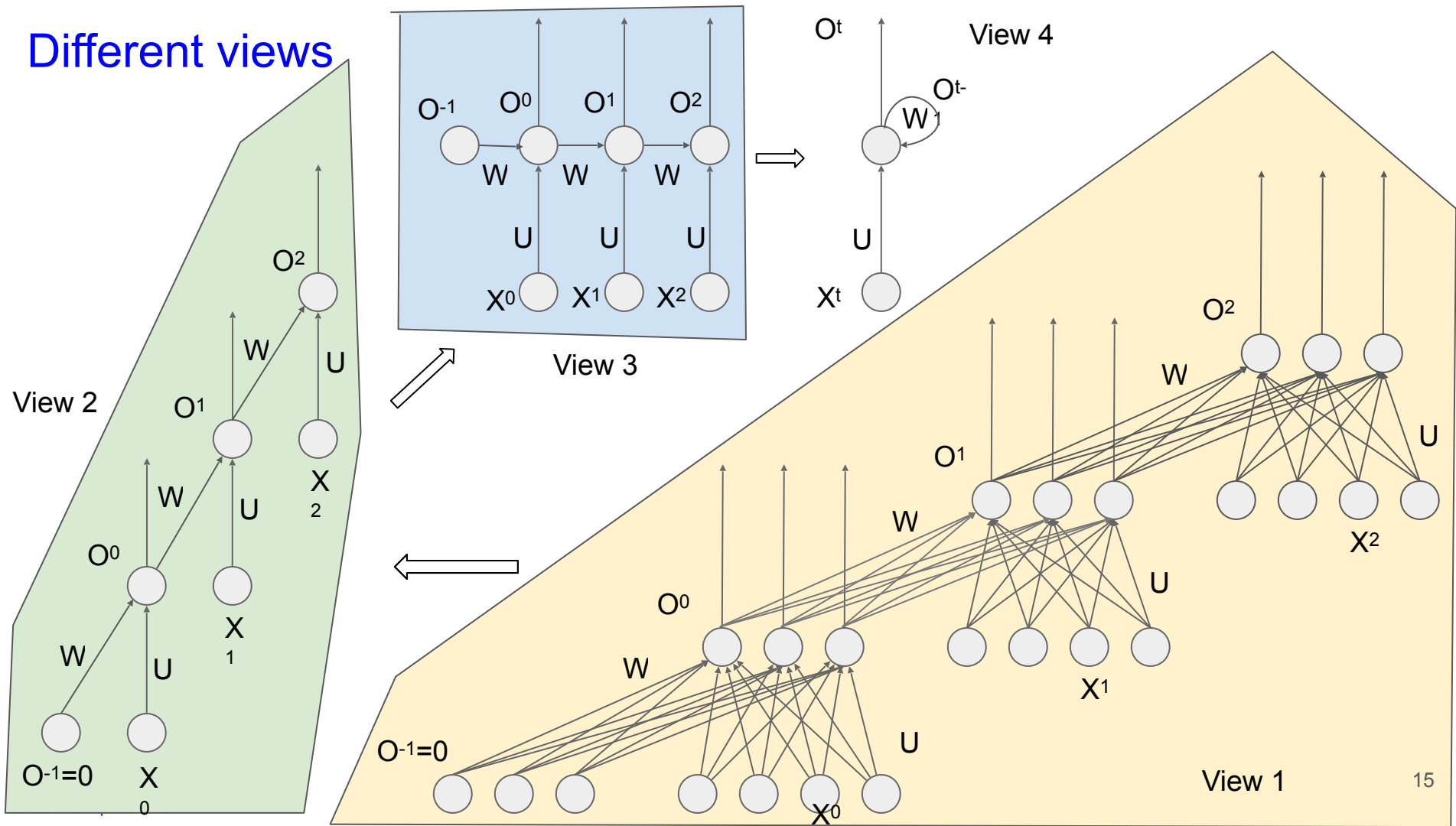
Different views of the network



Different views



Different views



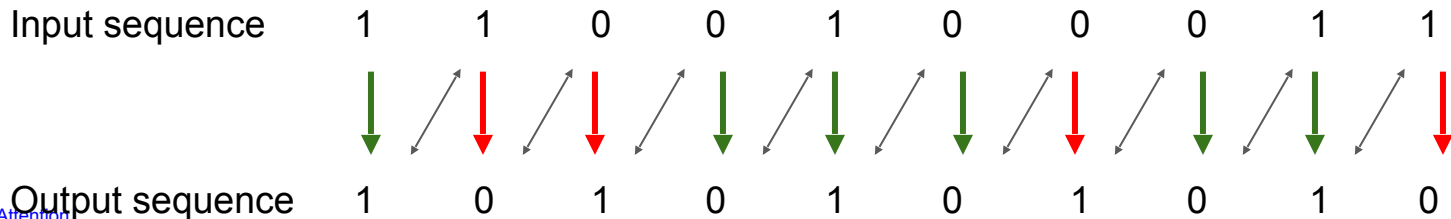
When to use RNNs

Usage

- Depends on the problems that we aim to solve.
- Typically good for sequence processing.
- Some sort of memorization is required.

Bit reverse problem

- Problem definition:
 - **Problem 1:** Reverse a binary digit.
 - $0 \rightarrow 1$ and $1 \rightarrow 0$
 - **Problem 2:** Reverse a sequence of binary digits.
 - $0\ 1\ 0\ 1\ 0\ 0\ 1 \rightarrow 1\ 0\ 1\ 0\ 1\ 1\ 0$
 - Sequence: Fixed or Variable length
 - **Problem 3:** Reverse a sequence of bits over time.
 - $0\ 1\ 0\ 1\ 0\ 0\ 1 \rightarrow 1\ 0\ 1\ 0\ 1\ 1\ 0$
 - **Problem 4:** Reverse a bit if the current i/p and previous o/p are same.



Data

Let

- **Problem 1**

- I/p dimension: **1 bit**

O/p dimension: **1 bit**

- **Problem 2**

- Fixed

- I/p dimension: **10 bit**

O/p dimension: **10 bit**

- Variable: Pad each sequence upto max sequence length: **10**

- Padding value: **-1**

- I/p dimension: **10 bit**

O/p dimension: **10 bit**

- **Problem 3 & 4**

- Dimension of each element of I/p (X)

: **1 bit**

- Dimension of each element of O/p (O)

: **1 bit**

- Sequence length

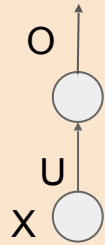
: **10**

Network Architecture

No. of I/p neurons = I/p dimension
No. of O/p neurons = O/p dimension

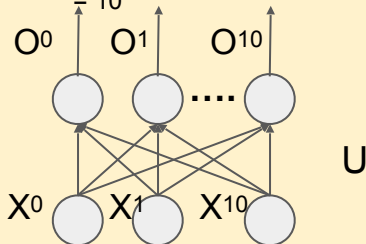
Problem 1:

- I/p neurons = 1
- O/p neurons = 1



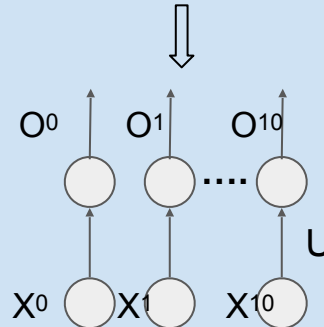
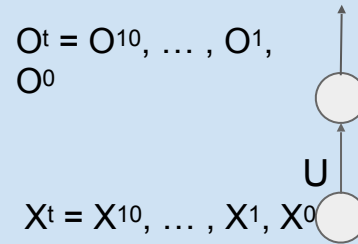
Problem 2: Fixed & Variable

- I/p neurons = 10
- O/p neurons = 10



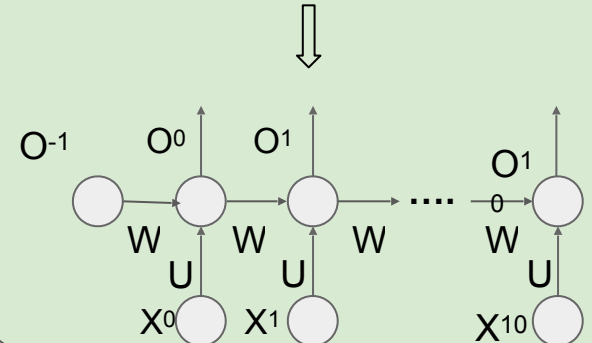
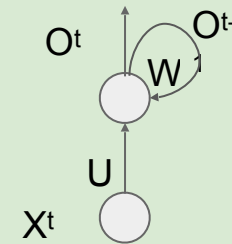
Problem 3:

- I/p neurons = 1
- O/p neurons = 1
- Seq len = 10



Problem 4:

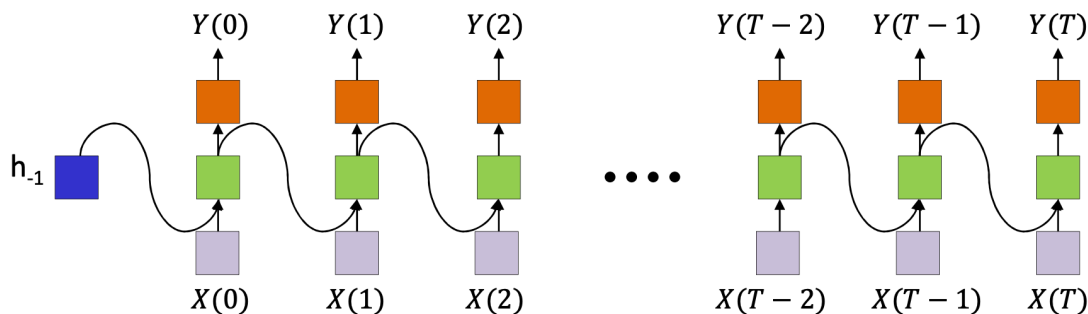
- I/p neurons = 1
- O/p neurons = 1
- Seq len = 10



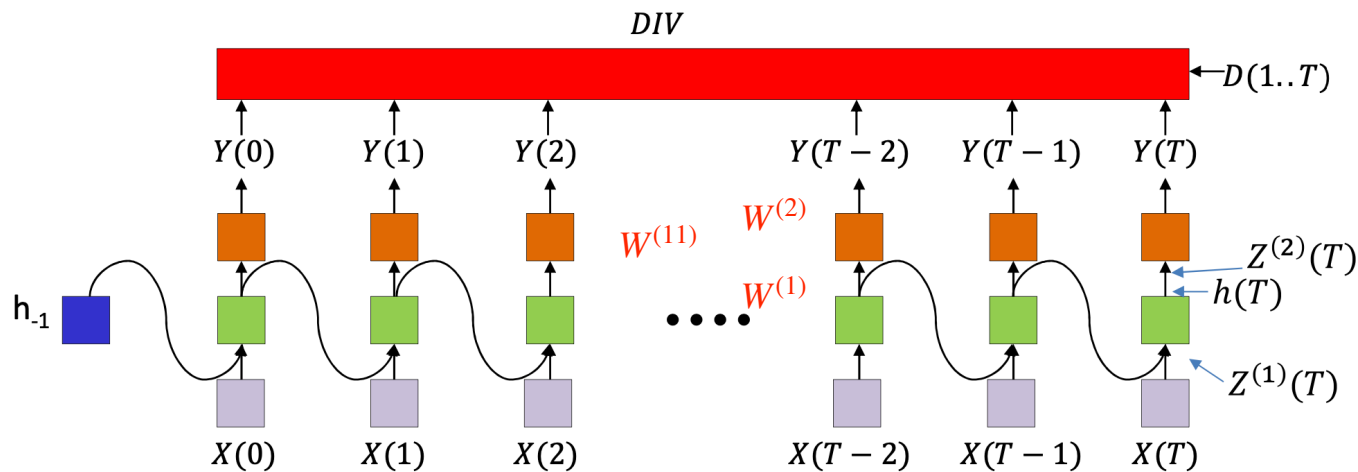
Training of RNNs

How to train RNNs?

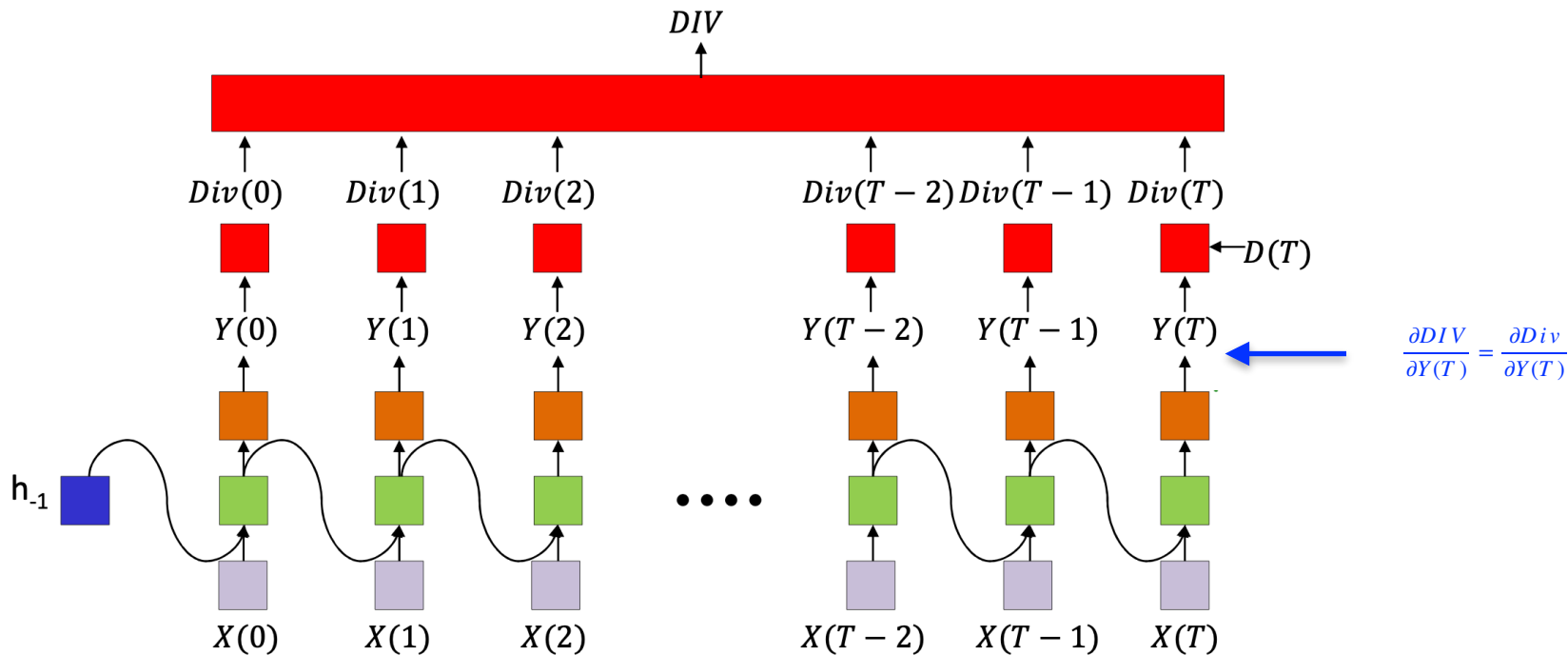
- Typical FFN
 - Backpropagation algorithm
- RNNs
 - A variant of backpropagation algorithm namely **Back-Propagation Through Time (BPTT)**.



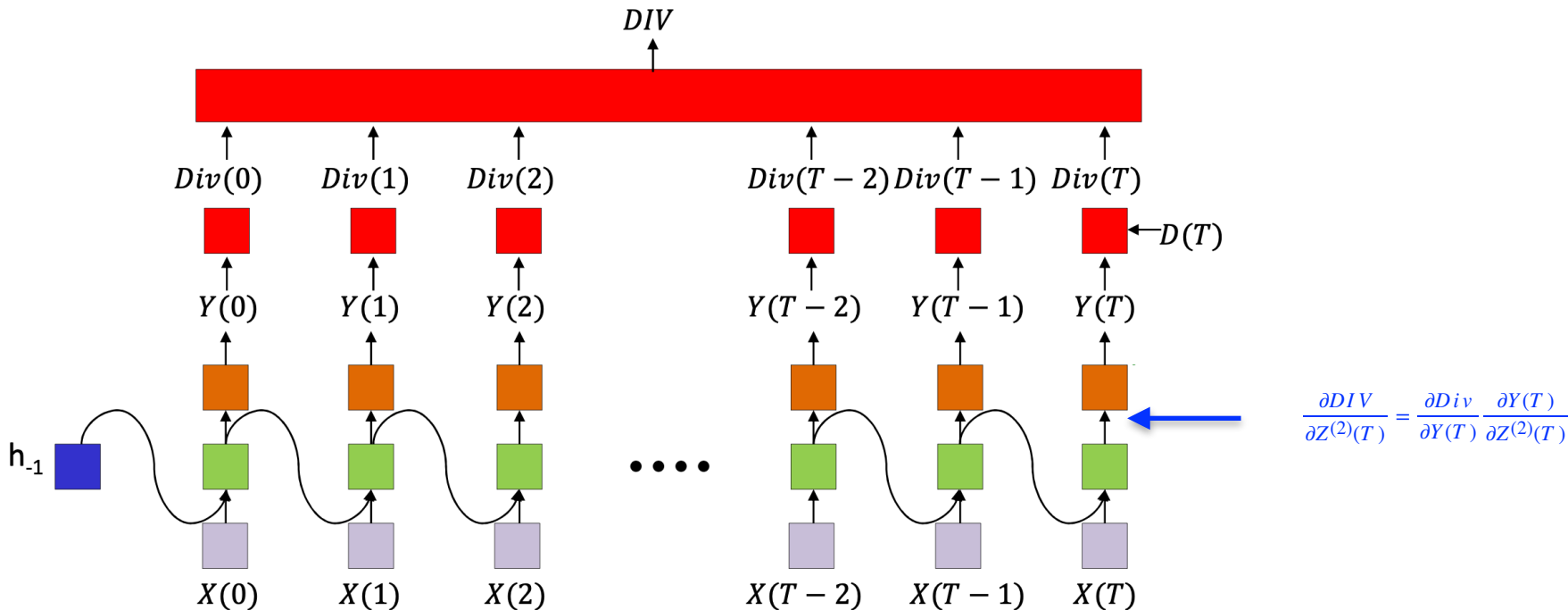
BackPropagation Through Time (BPTT)



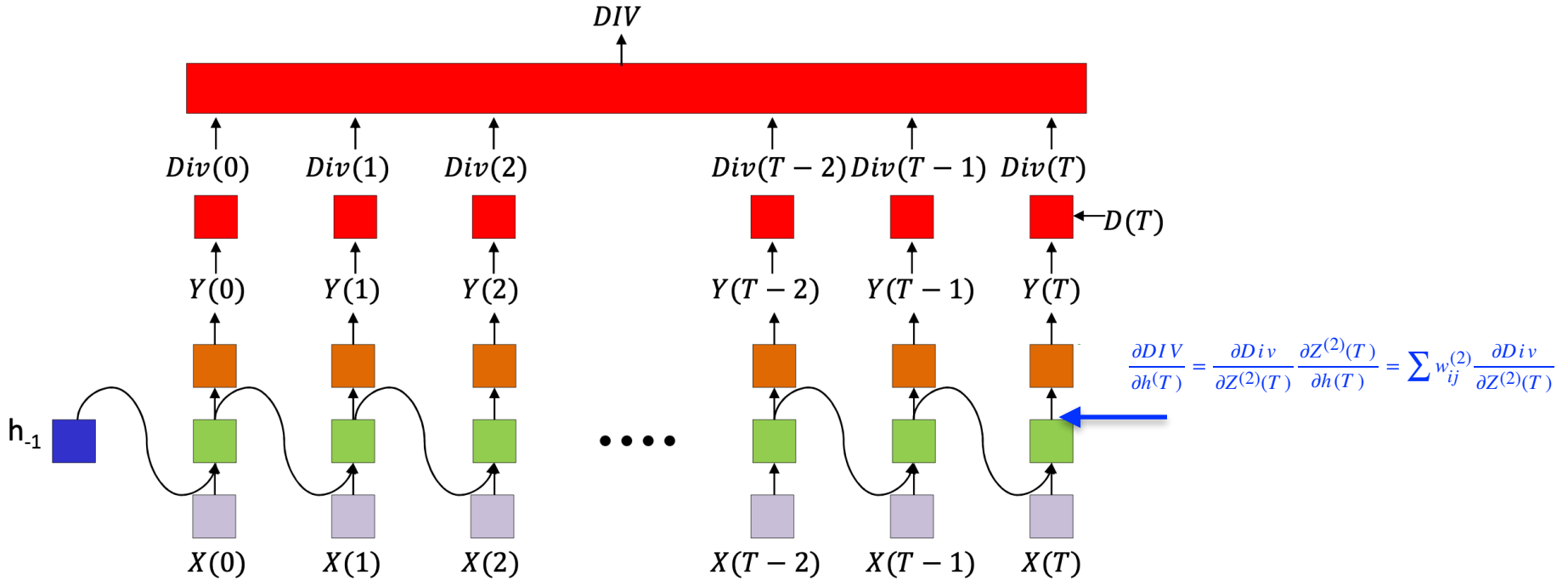
BackPropagation Through Time (BPTT)



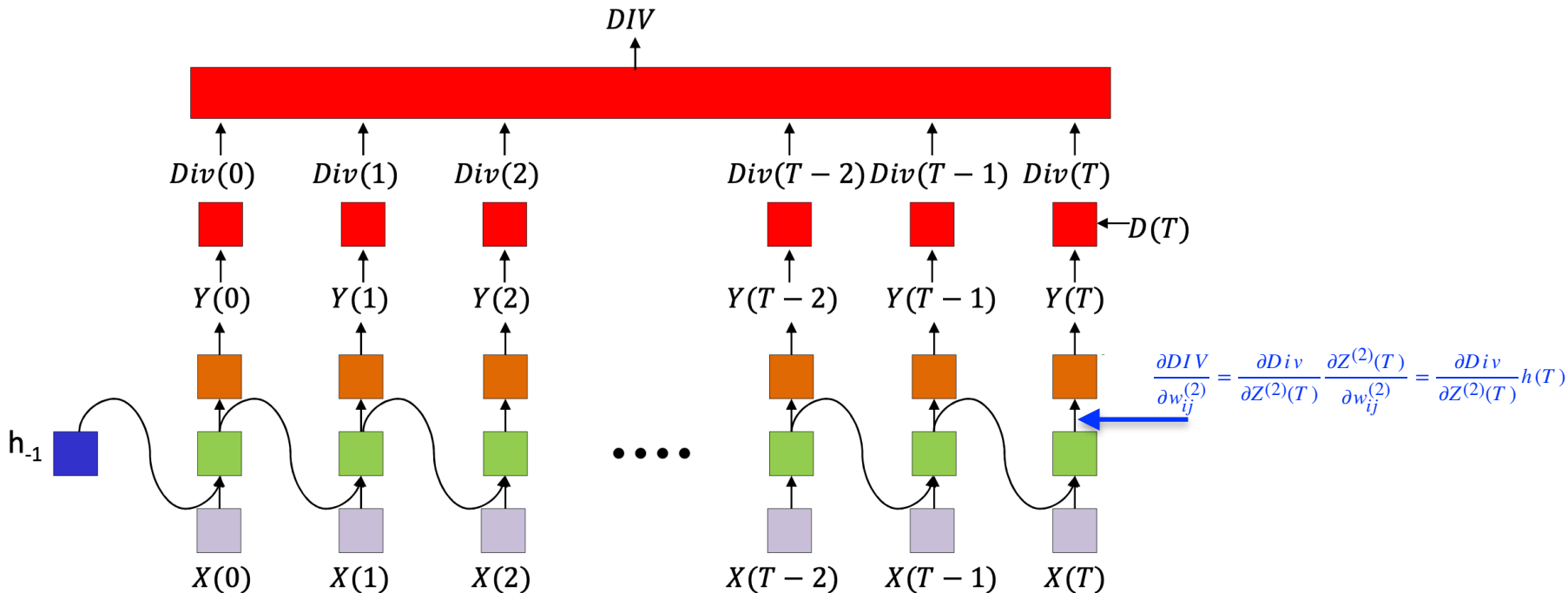
BackPropagation Through Time (BPTT)



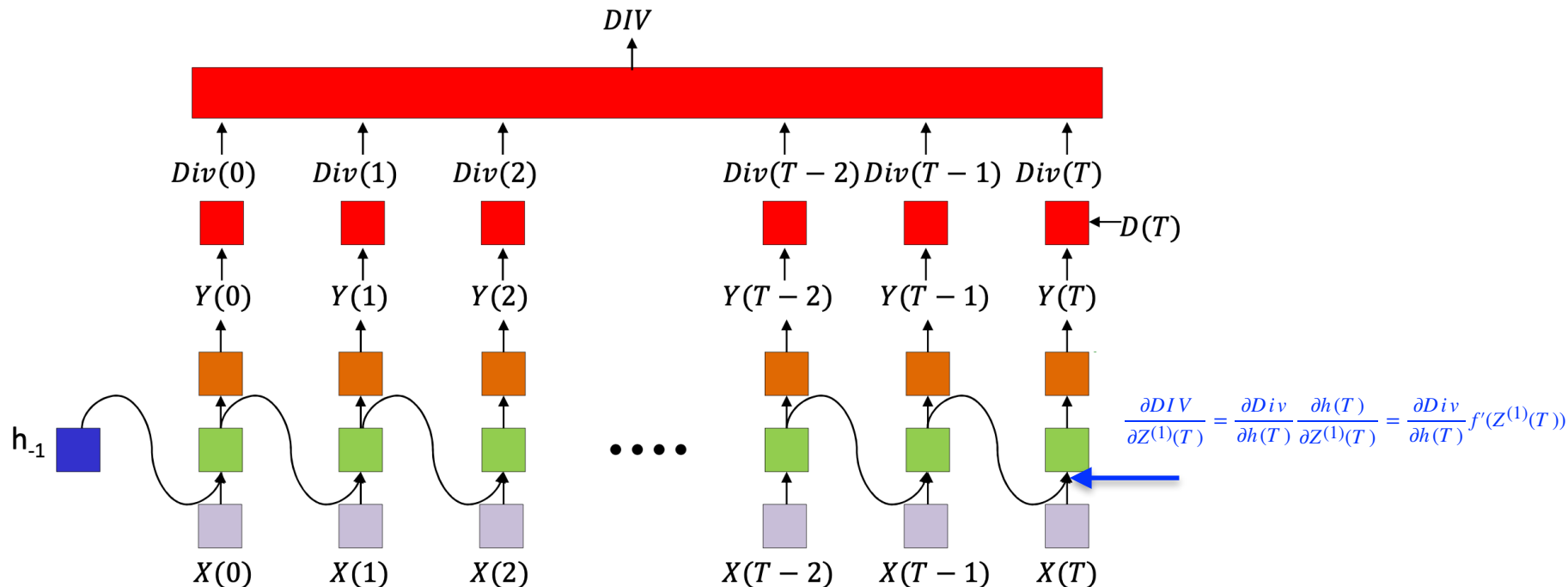
BackPropagation Through Time (BPTT)



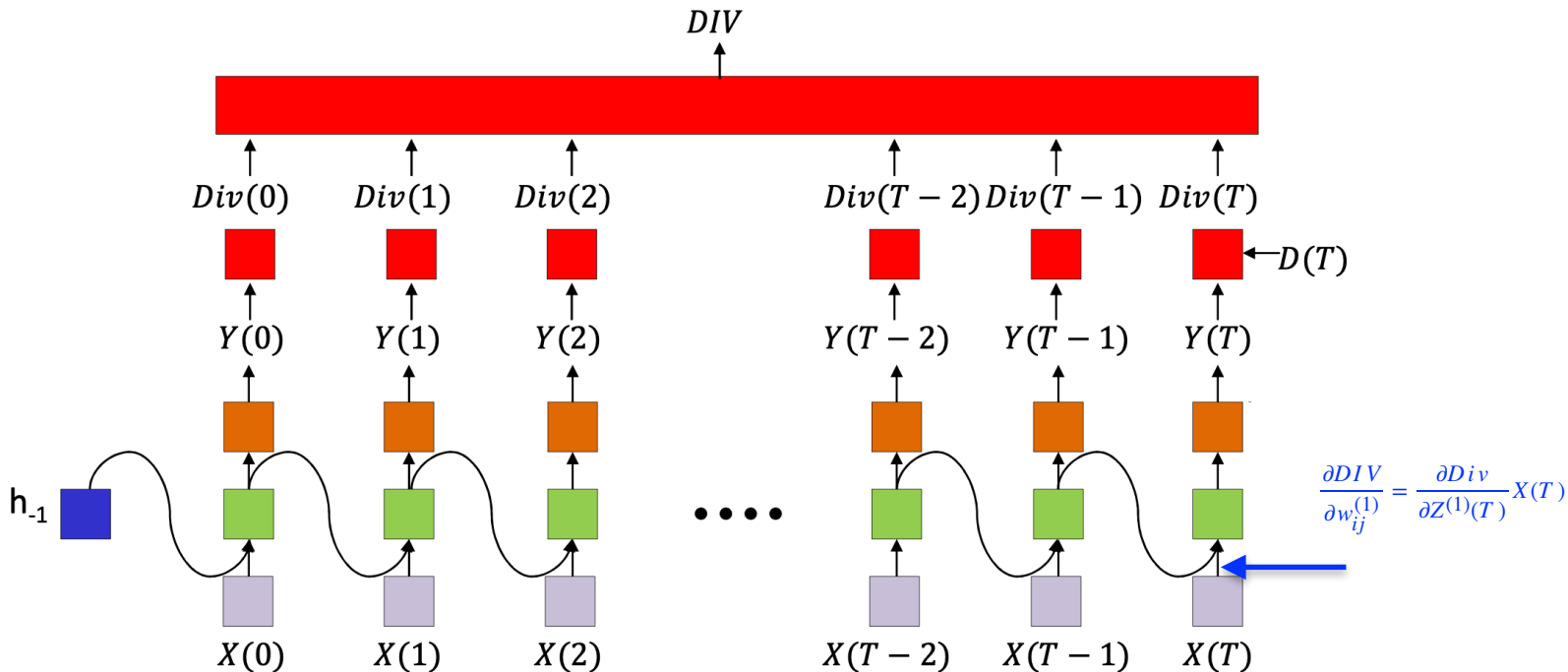
BackPropagation Through Time (BPTT)



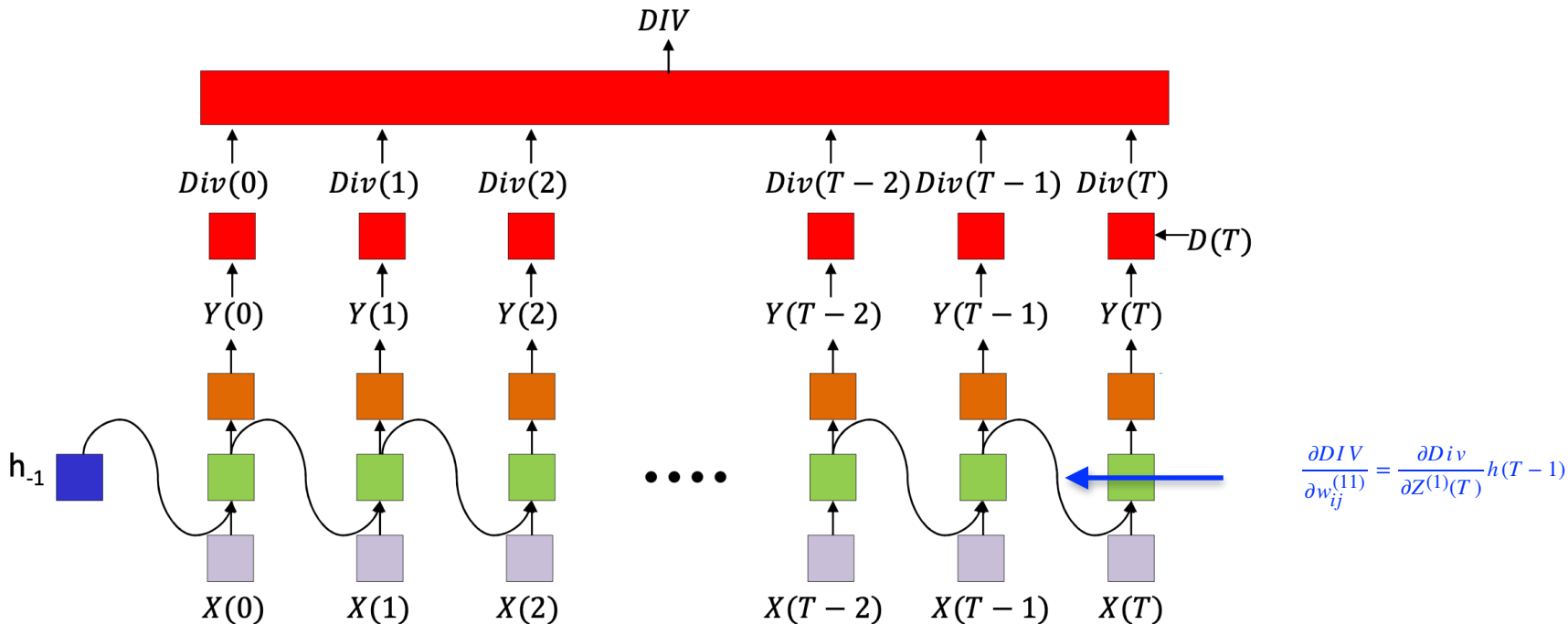
BackPropagation Through Time (BPTT)



BackPropagation Through Time (BPTT)

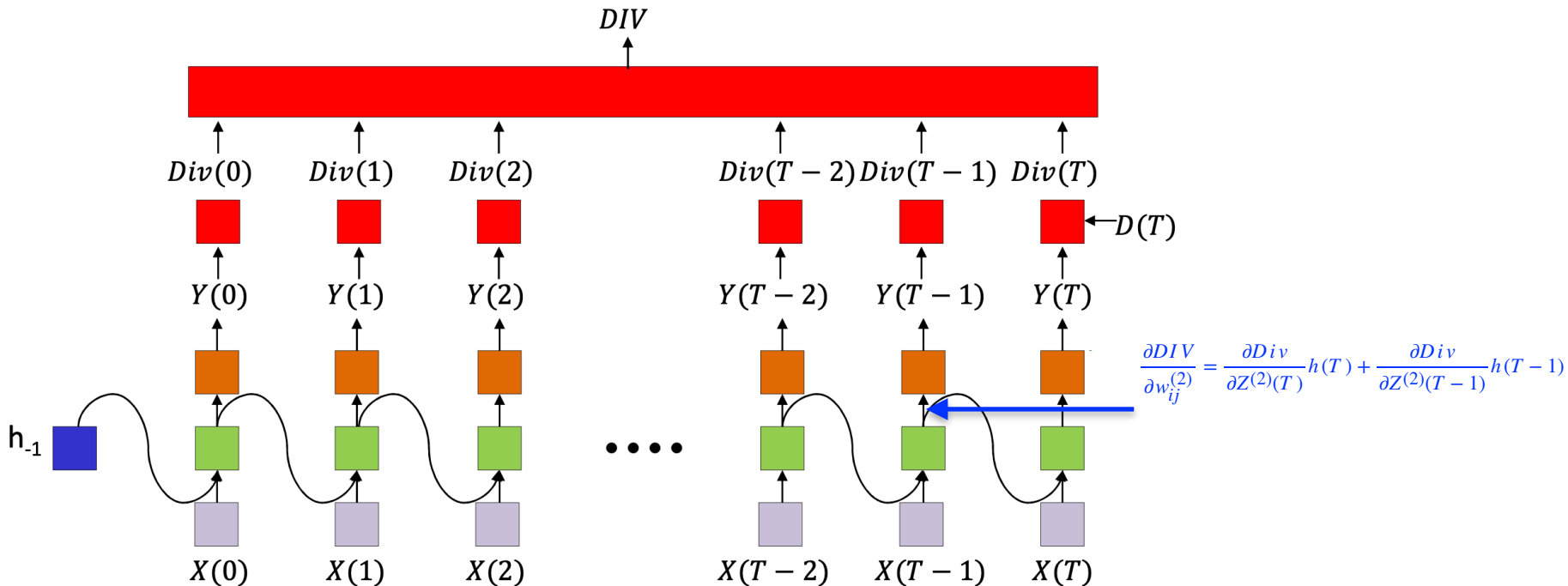


BackPropagation Through Time (BPTT)

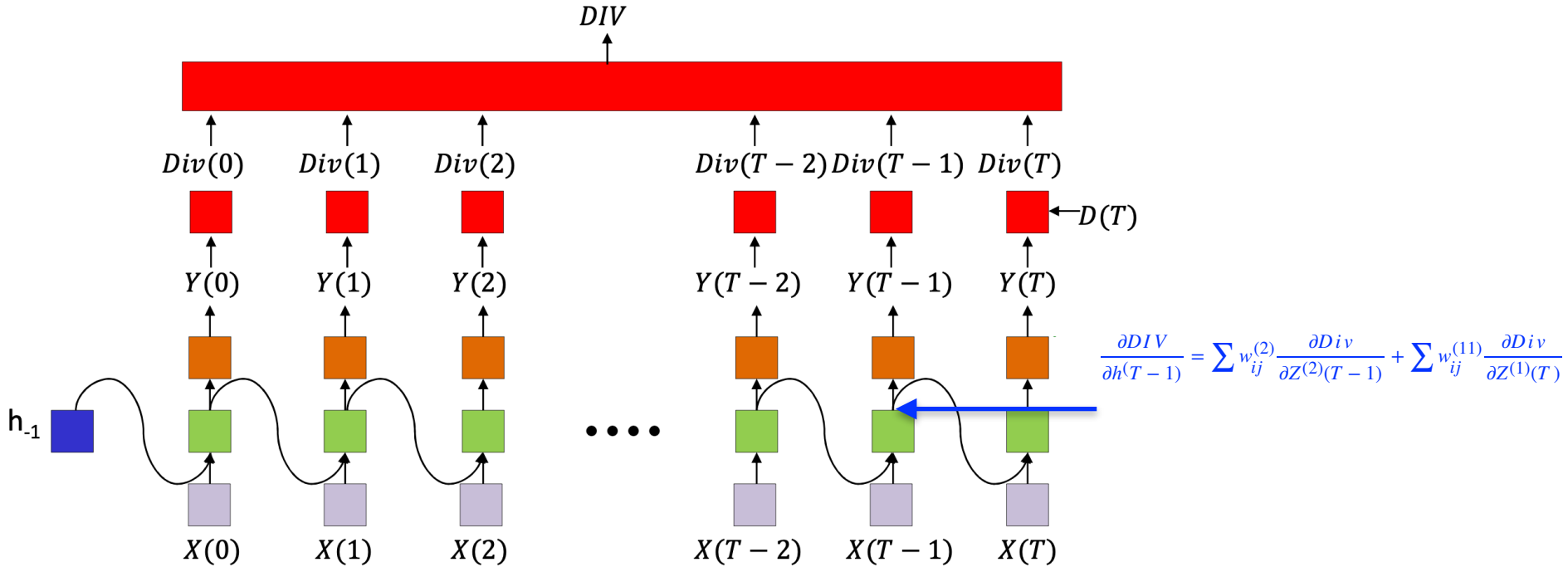


$$\frac{\partial DIV}{\partial w_{ij}^{(11)}} = \frac{\partial Div}{\partial Z^{(1)}(T)} h(T-1)$$

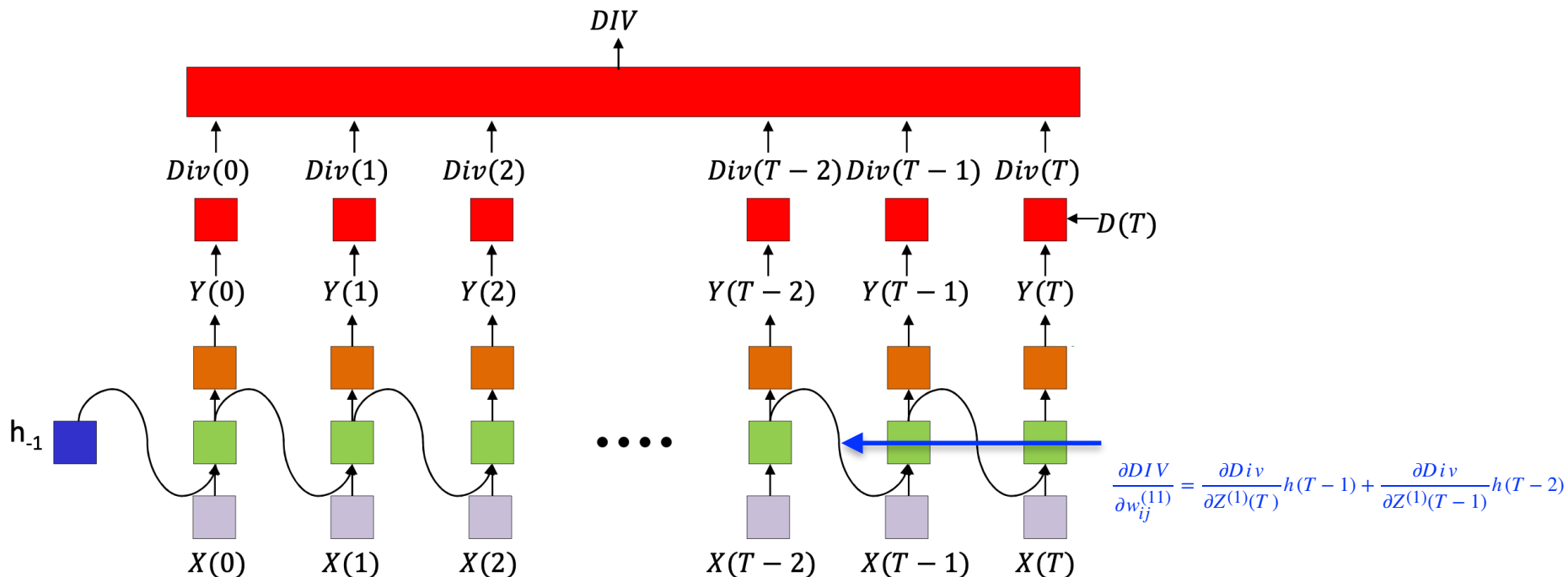
BackPropagation Through Time (BPTT)



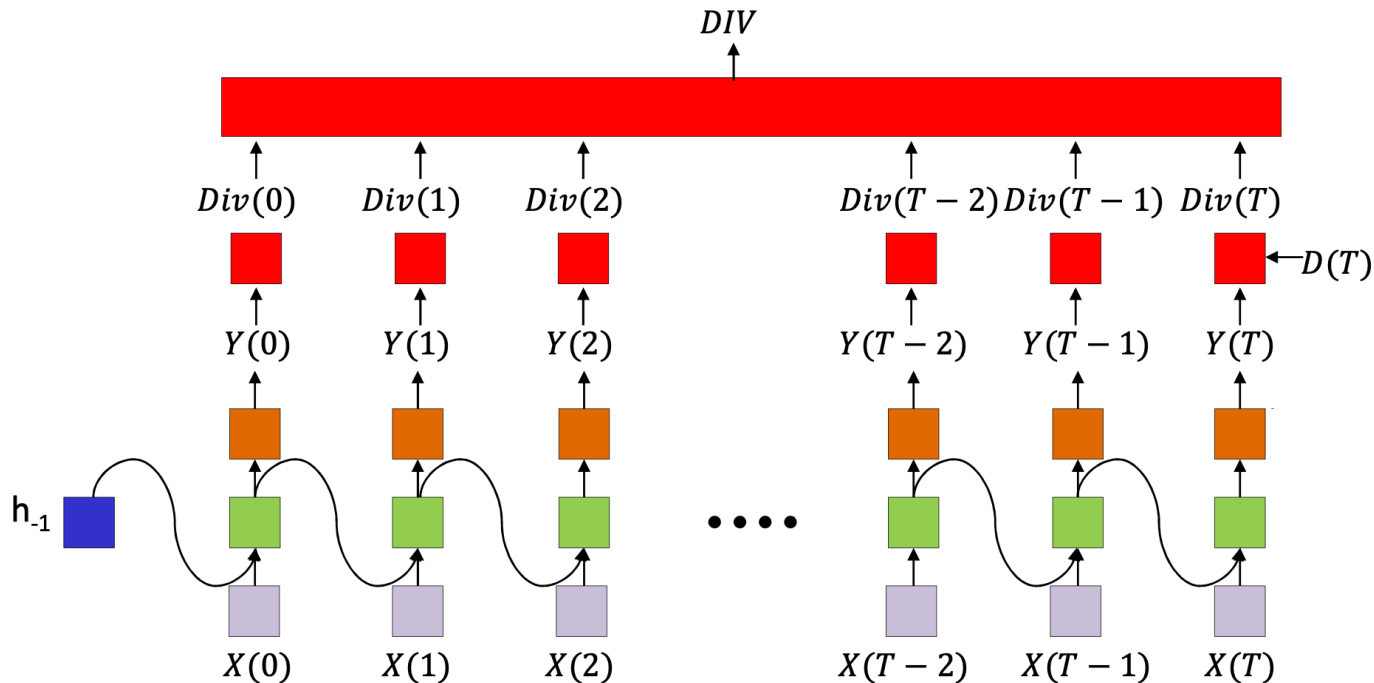
BackPropagation Through Time (BPTT)



BackPropagation Through Time (BPTT)



BackPropagation Through Time (BPTT)

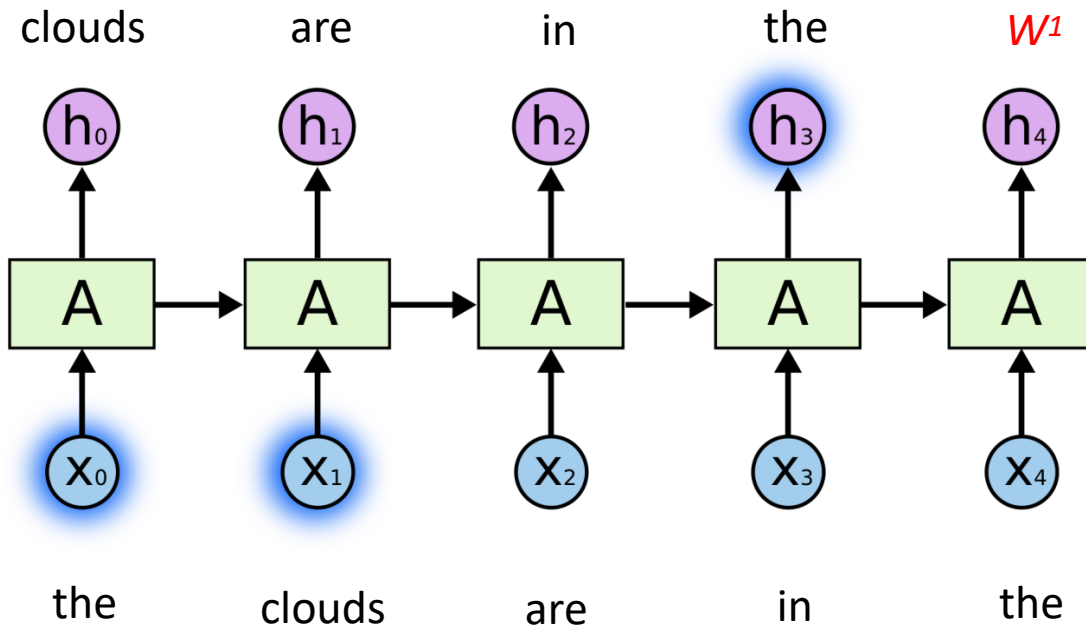


Through each time step.

Problems with RNNs

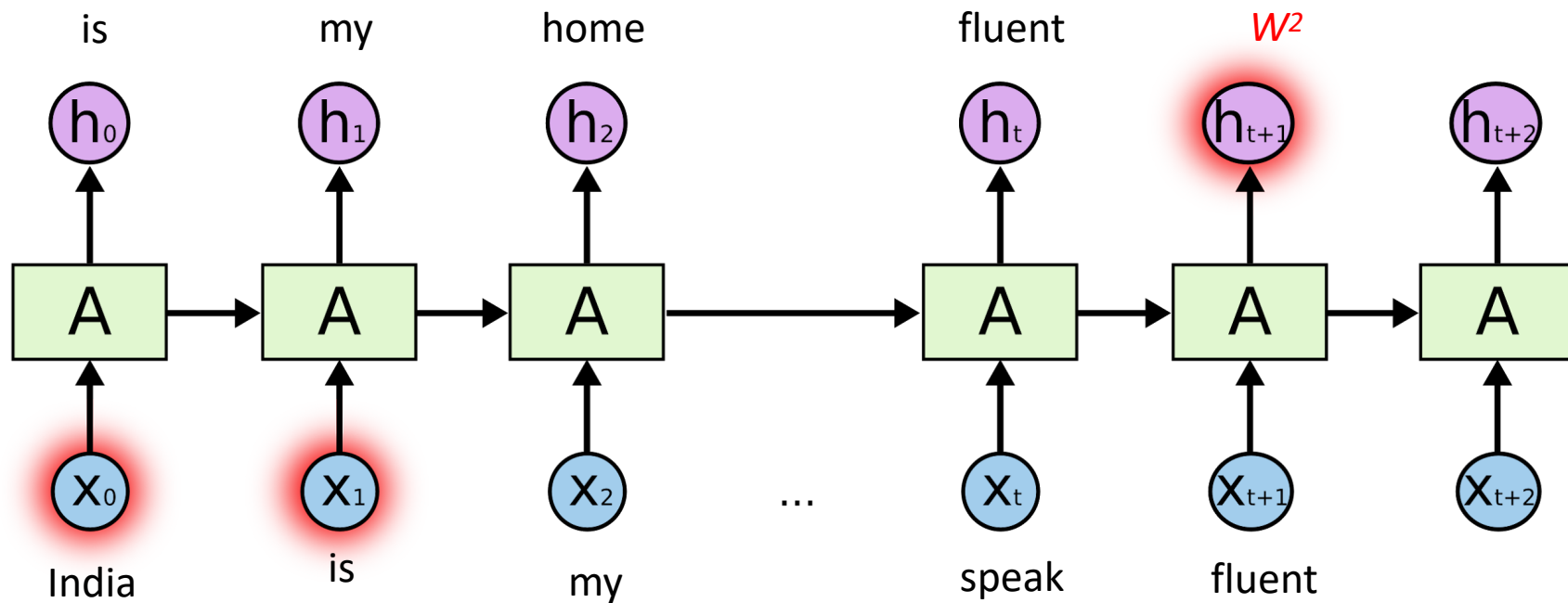
Language modelling: Example - 1

- “the clouds are in the *sky*”



Language modelling: Example - 2

- “India is my home country. I can speak fluent *Hindi*.”

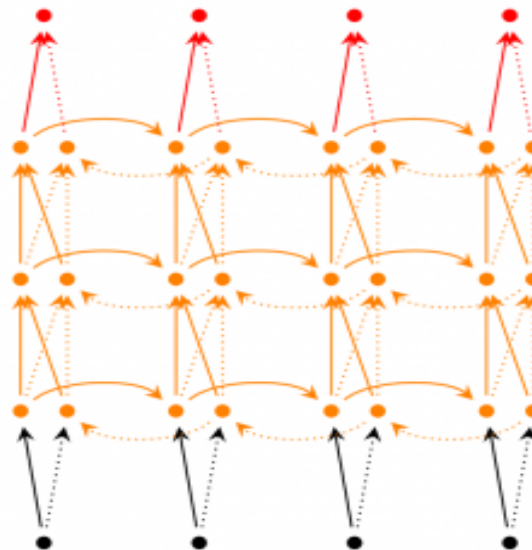
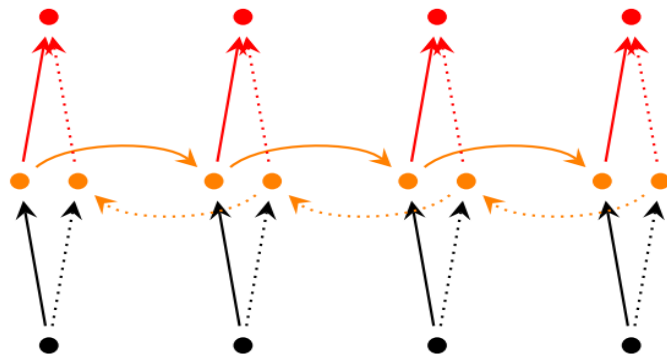


Vanishing/Exploding gradients

- Cue word for the prediction
 - Example 1: *sky* → *clouds* [3 units apart]
 - Example 2: *hindi* → *India* [9 units apart]
- As the sequence length increases, it becomes hard for RNNs to learn “long-term dependencies.”
 - **Vanishing gradients:** If weights are small, gradient shrinks exponentially. Network stops learning.
 - **Exploding gradients:** If weights are large, gradient grows exponentially. Weights fluctuate and become unstable.

RNN extensions

- Bi-directional RNN
- Deep (Bi-directional) RNN



Long Short Term Memory (LSTM)

Hochreiter & Schmidhuber (1997)

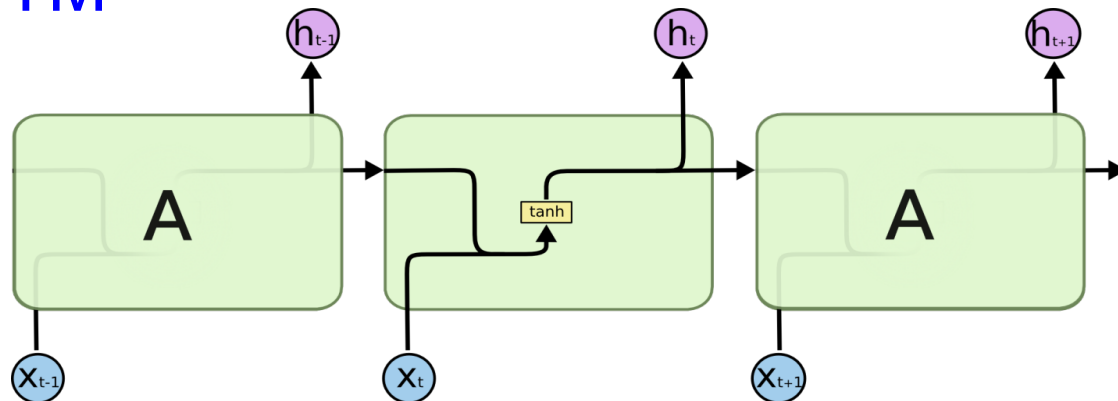
<https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory>

LSTM

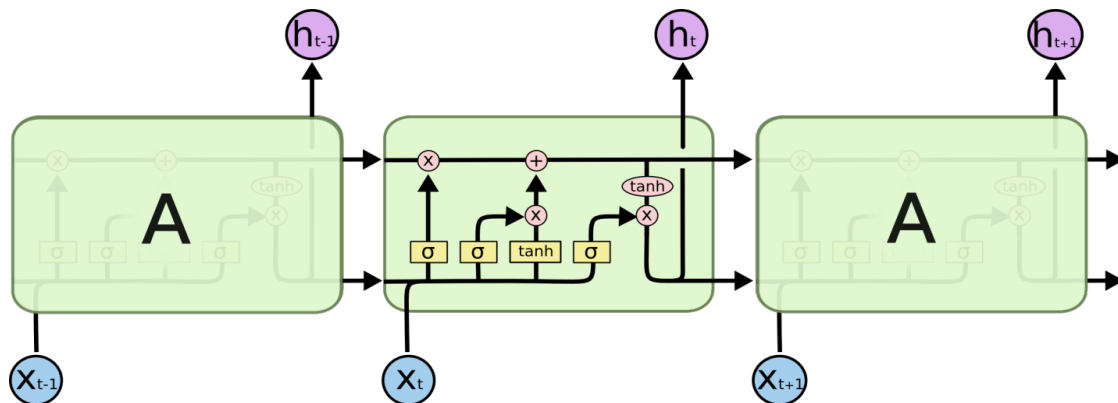
- A variant of simple RNN (Vanilla RNN)
- Capable of learning long dependencies.
- Regulates information flow from recurrent units.

Vanilla RNN vs LSTM

Vanilla RNN cell



LSTM cell



An LSTM cell

- Cell state c_t (blue arrow), hidden state h_t (green arrow) and input x_t (red arrow)
- Three gates

- Forget (Red-dotted box)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t])$$

$$c_t = f_t \otimes c_{t-1}$$

- Input (Green-dotted box)

$$z_t = \tanh(W_z \cdot [h_{t-1}, x_t])$$

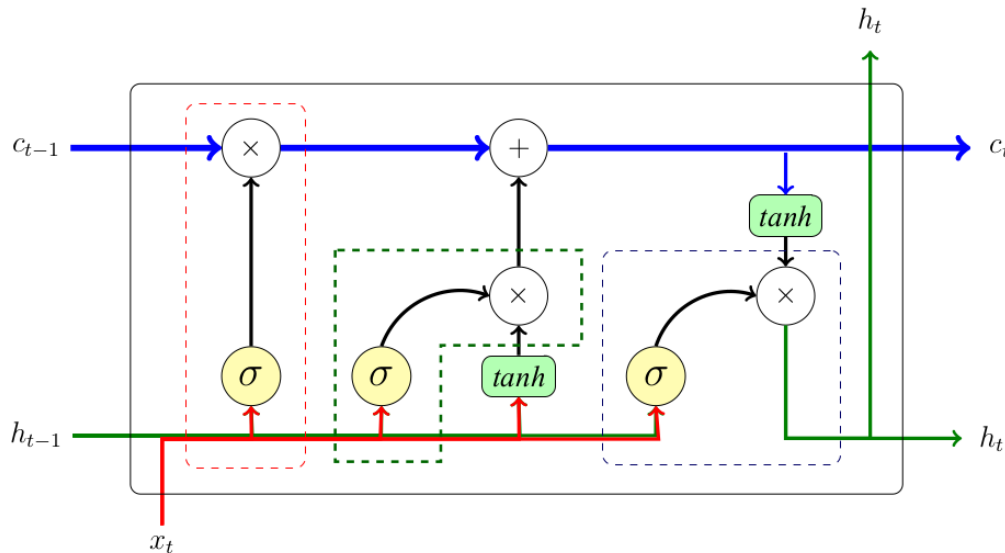
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t])$$

$$c_t = c_t + (i_t \otimes z_t)$$

- Output (Blue-dotted box)

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t])$$

$$h_t = o_t \otimes \tanh(c_t)$$



An LSTM cell - Peepholing

- Cell state c_t (blue arrow), hidden state h_t (green arrow) and input x_t (red arrow)
- Three gates

- Forget (Red-dotted box)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t])$$

$$c_t = f_t \otimes c_{t-1}$$

- Input (Green-dotted box)

$$z_t = \tanh(W_z \cdot [h_{t-1}, x_t])$$

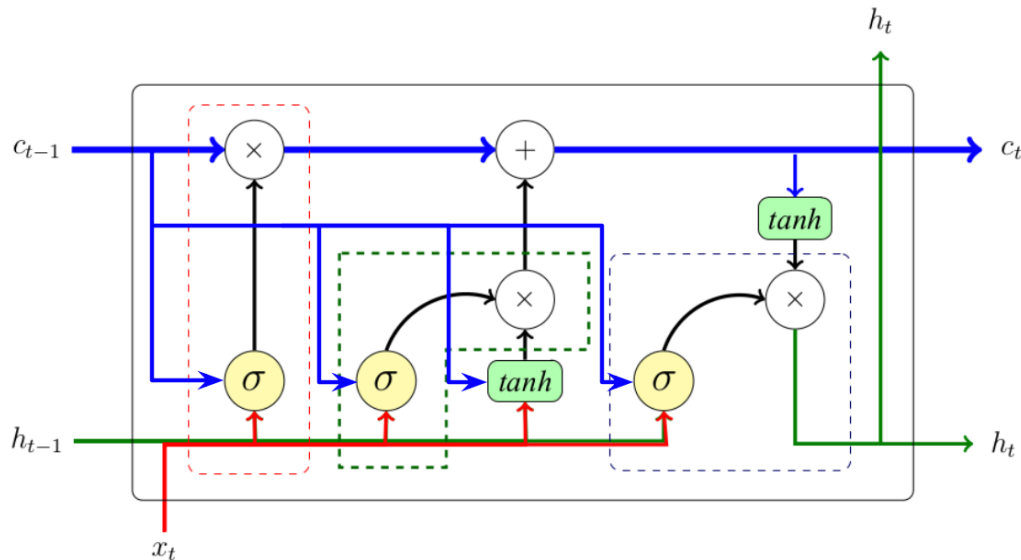
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t])$$

$$c_t = c_t + (i_t \otimes z_t)$$

- Output (Blue-dotted box)

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t])$$

$$h_t = o_t \otimes \tanh(c_t)$$



Gated Recurrent Units (GRU)

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, Yoshua Bengio (2014)

Gated Recurrent Unit (GRU) [Cho et al. [\(2014\)](#)]

- A variant of simple RNN (Vanilla RNN)
- Similar to LSTM
 - Whatever LSTM can do GRU can also do.
- Differences
 - Cell state and hidden are merged together
 - Two gates
 - Reset gate - similar to forget
 - Update gate - similar to input gate
 - No output gate
 - Cell/Hidden state is completely exposed to subsequent units.
- GRU needs fewer parameters to learn and is relatively efficient *w.r.t.* computation.

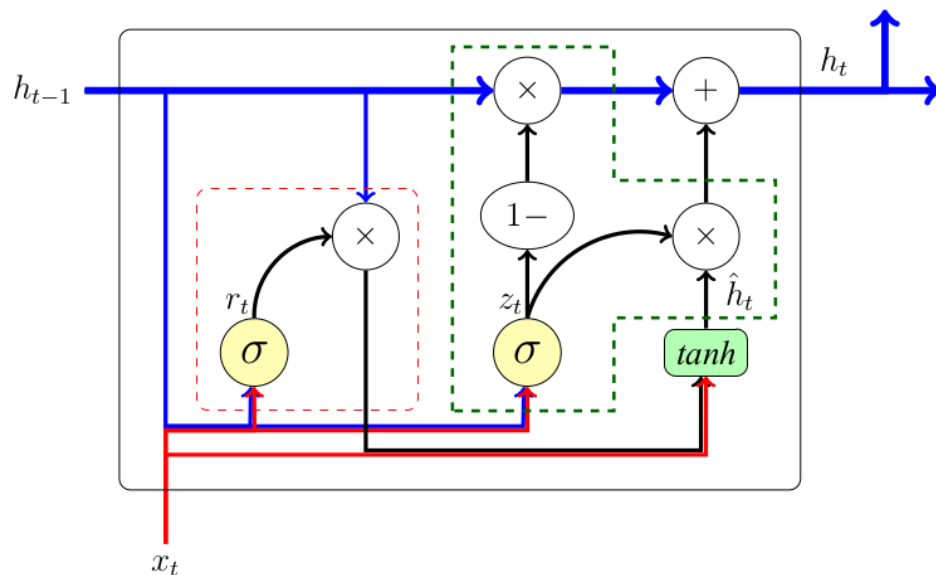
A GRU cell

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

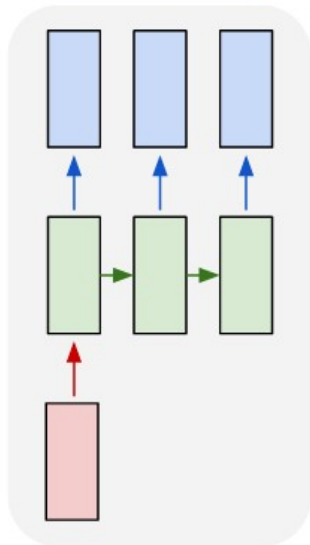
$$\hat{h}_t = \tanh(W \cdot [r_t \otimes h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \otimes h_{t-1} + z_t \otimes \hat{h}_t$$



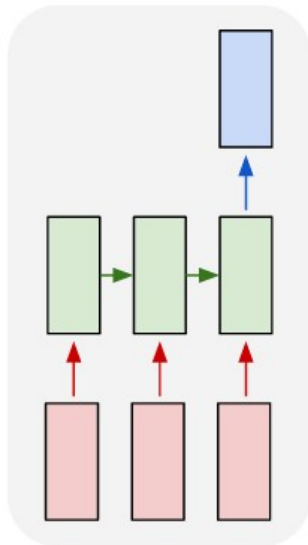
Different configurations of RNNs

one to many



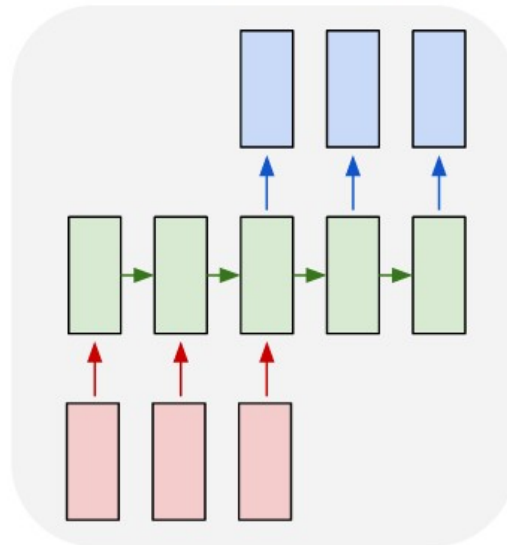
**Image
Captioning**

many to one



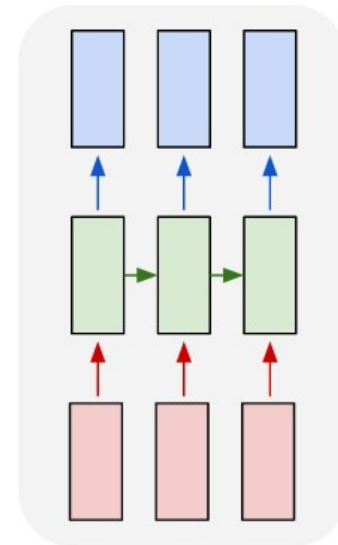
**Sentiment
Analysis**

many to many



**Machine
Translation**

many to many

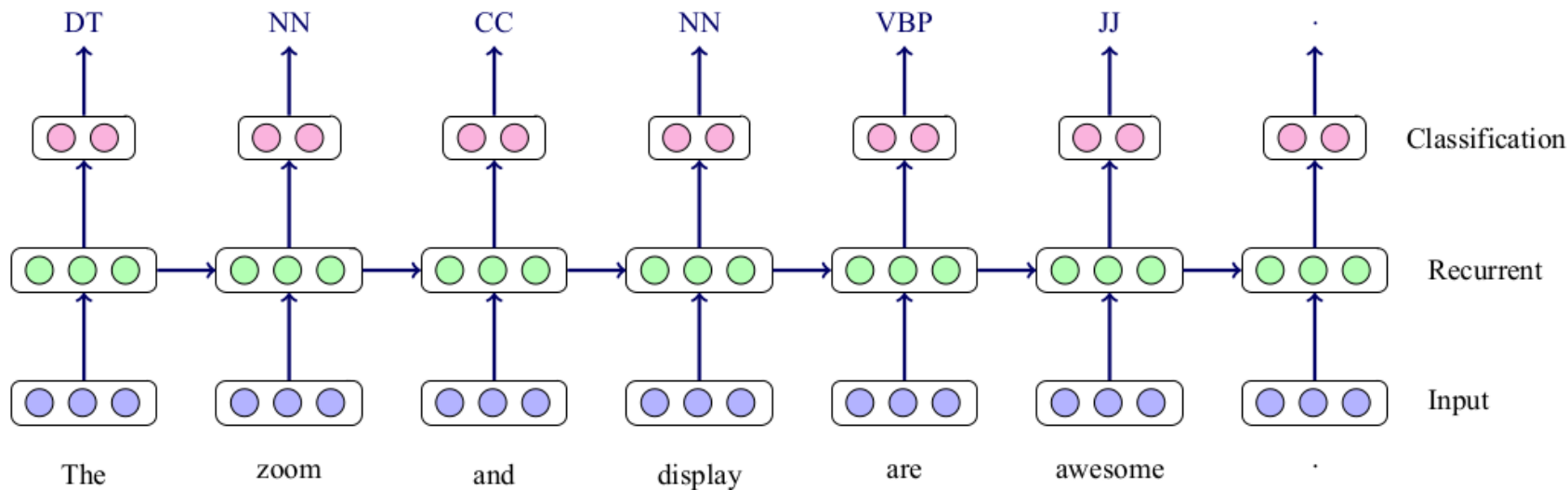


**Language
modelling**

RNN/LSTM/GRU for Sequence Labelling

Part-of-speech tagging:

- Given a sentence X, tag each word its corresponding grammatical class.



RNN/LSTM/GRU for Sentence Classification

Sentiment Classification:

- Given a sentence X, identify the expressed sentiment.

